

OENEYE Text Emoticon & Typographic Brand Tokens

This design guidelines document outlines the constraints, formatting options, and technical capabilities of text-based emoticons drafted for the **OENEYE** brand identity, translating its iconic infinity eye concept into pure string tokens.

1. The Primary Identity Token

The compact token map explicitly mirrors the structure of the graphical logo (a stylized infinity loop doubling as an open eye and a winking eye):

0∞;

- 0** (Zero) represents the left open eye (pupil, iris, and wide socket).
- ∞** (Infinity Symbol, U+221E) forms the continuous central interconnected geometry.
- ;** (Semicolon) provides the punctuation stack to imply the right eye's curved winking eyelid.

2. Complete Typographic Options Matrix

Token / Option	Type	Visual Concept / Interpretation	Platform Compatibility
0∞;	Single-Token Unicode	Open eye + Infinity core + Wink. Direct literal mapping to the core visual identity.	Highly reliable. Requires font coverage for the math infinity character.
o∞o	Single-Token Unicode	Symmetrical loops with flanking lowercase eyes surrounding the infinity bridge.	Highly reliable. Works well in proportional and monospace layouts.
o∞ob	Single-Token with Thumb	The symmetrical brand signature integrated with a lowercase b acting as an upward thumb.	High readability in modern text environments.
(o) (o)b	Pure ASCII	Two circular eyes side-by-side flanked by a standard text thumb indicator.	100% universal. Bypasses all Unicode limitations completely.

3. Capabilities & Constraints (The "Can vs. Can't" Rules)

What these text tokens CAN do:

- **Universal Distribution:** Operates natively in environment pipelines where images or custom attachments are blocked or restricted (e.g., source code comments, Git commits, text-based terminal summaries, and email signatures).
- **Zero Performance Footprint:** Loads instantly with zero network asset dependency or external hosting overhead.
- **Developer & Community Alignment:** Acts as an engaging text-based signature or easter egg throughout documentation and command-line interfaces.

What these text tokens CANNOT do:

- **Guarantee Monospace Layout Symmetry:** In variable-width (proportional) text streams, specific letter variations like `0` or `o` might shift unevenly next to `∞` depending on structural rendering engines.
- **Enforce Color Specifications:** Text strings strictly inherit the native ambient font colors and text-scaling boundaries of their container. They cannot force localized corporate colors natively.
- **Automate Graphic Conversions:** External software frameworks (e.g., standard chat interfaces) handle text literally and will not convert these specific unique tokens into high-fidelity graphics without custom macro hooks.